

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Activity / Exp No.: Project-Based Learning (PBL) Part 1

Student Name: J.Srikar

Roll No.: A24126552083

Date: 21-09-2026

1. TITLE

Project-Based Learning (PBL) part 1

Node Package Manager (NPM) Workflow, Package Manifest Configuration, and Chalk Module Integration

2. AIM

To initialize a Node.js project manifest (package.json), install and manage third-party external dependencies from the npm registry using the Node Package Manager (NPM), configure ECMAScript Modules ('type': 'module'), and implement colored terminal console styling using the Chalk library.

3. OBJECTIVE

- To initialize a project manifest file (package.json) using `npm init -y` with standard project metadata.
- To install external third-party dependencies from the global npm registry using the command `npm install chalk`.
- To understand the role and structure of package.json, package-lock.json, and the local node_modules directory.
- To configure modern ECMAScript Module (ESM) syntax by setting 'type': 'module' in package.json.
- To define custom script execution shortcuts under the 'scripts' manifest block ('start': 'node app.js').
- To import third-party ESM packages into application code using the modern `import ... from '...'` statement.
- To apply terminal ANSI styling and colored logging output using `chalk.green()`, `chalk.blue()`, and `chalk.yellow()`.

4. THEORY

The Node Package Manager (NPM) is the world's largest software registry and package management ecosystem for JavaScript and Node.js. It comprises three distinct components: the npm cloud registry (a public online repository of open-source libraries), the command-line interface (CLI tool bundled with Node.js), and local project dependency management. NPM empowers developers to discover, install, update, and manage modular external libraries, accelerating full-stack web application development while maintaining consistent versioning across project environments.

Every modern Node.js project is defined by its manifest file, package.json, which records project metadata (name, version, description), execution scripts, entrypoints, and third-party dependencies. When a library is installed via `npm install`, npm records the dependency under 'dependencies' with semantic versioning ranges (e.g., `^5.6.2`, where the caret permits non-breaking minor and patch updates). Concurrently, npm generates package-lock.json, which records an exact, deterministic dependency tree.

4. THEORY (Continued)

Historically, Node.js relied exclusively on the CommonJS (CJS) module system utilizing `require()` and `module.exports`. ECMAScript 2015 introduced official ECMAScript Modules (ESM) featuring static import and export statements. By adding `'type': 'module'` to `package.json`, developers instruct Node.js to treat all `.js` files in the project directory as ESM modules, permitting native top-level imports and seamless integration with modern libraries like Chalk (version 5 and above is purely ESM).

Terminal emulators support ANSI escape sequences—special character codes that control text color, background styling, and font attributes in the console. The Chalk library abstracts these low-level byte sequences into a readable, chainable JavaScript API. Chalk dynamically detects terminal color support (16-color, 256-color, TrueColor, or non-TTY piped streams) and outputs vibrant, color-coded strings, drastically improving CLI readability and developer feedback.

5. ALGORITHM / PROCEDURE

1. Open a terminal and navigate to the project directory (PBL/NPM).
2. Verify Node.js and NPM installations using `node -v` and `npm -v` in the terminal.
3. Initialize a new Node.js package manifest by executing `npm init -y`.
4. Open `package.json` and verify initial metadata fields (name, version, scripts).
5. Add `'type': 'module'` to `package.json` to enable ECMAScript Module (ESM) syntax.
6. Configure an execution script shortcut by adding `'start': 'node app.js'` under `'scripts'`.
7. Install the Chalk package from the npm registry using the command `npm install chalk`.
8. Verify that `package.json` updates with chalk under dependencies, and `package-lock.json` is created.
9. Create the main JavaScript application file named `app.js`.
10. Import the Chalk library into `app.js` using the ESM import statement: `import chalk from 'chalk'`.
11. Implement a green welcome message using `console.log(chalk.green('Hello! Welcome to Node.js.'))`.
12. Implement a blue message using `console.log(chalk.blue('This message is displayed using Chalk.'))`.
13. Implement a yellow message using `console.log(chalk.yellow('NPM successfully installed external package.'))`.
14. Execute the project using the command `npm start` (or `node app.js`).
15. Verify that the terminal displays the three colorized strings in green, blue, and yellow without syntax errors.

6. PROGRAM

package.json

```
{
  "name": "npm-demo",
  "version": "1.0.0",
  "description": "NPM package example",
  "main": "app.js",
  "type": "module",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "chalk": "^5.6.2"
  }
}
```

app.js

```
import chalk from "chalk";

// Display messages using the Chalk package
console.log(chalk.green("Hello! Welcome to Node.js."));
console.log(chalk.blue("This message is displayed using Chalk."));
console.log(chalk.yellow("NPM successfully installed the external package."));
```

7. CODE EXPLANATION

Code Snippet	Detailed Technical Explanation
<code>"name": "npm-demo", "version": "1.0.0"</code>	Defines project unique identity and initial semantic release version in the npm ecosystem.
<code>"main": "app.js"</code>	Specifies the primary entrypoint execution script for the Node.js application package.
<code>"type": "module"</code>	Instructs Node.js V8 runtime to treat all project JavaScript files as ECMAScript Modules (ESM).
<code>"scripts": { "start": "node app.js" }</code>	Registers a convenient CLI command alias runnable via 'npm start' to trigger application execution.
<code>"dependencies": { "chalk": "^5.6.2" }</code>	Declares Chalk library as a production dependency with caret allowing compatible minor/patch upgrades.
<code>import chalk from "chalk";</code>	Imports default styling object from the Chalk package installed in the local node_modules directory.
<code>chalk.green("Hello!...")</code>	Applies ANSI escape sequence for green color (32m) to string literal and resets formatting (39m).
<code>chalk.blue("This message...")</code>	Applies ANSI escape sequence for blue color (34m) to string literal for clear informational distinction.
<code>chalk.yellow("NPM successfully...")</code>	Applies ANSI escape sequence for yellow color (33m) to render confirmation text.
<code>console.log(...)</code>	Transmits the ANSI-formatted, colorized strings to standard output (process.stdout) for terminal rendering.

8. TEST CASES AND EXECUTION DATA

Test Case	Target	Input / Action	Description	Expected Result	Actual Result	Status
TC-01	Environment	<code>node -v & npm -v</code>	Verify Node.js & NPM CLI installations	Version numbers displayed cleanly	Node v25.6.1 & npm 11.9.0 verified	Pass
TC-02	Manifest	<code>npm init -y</code>	Generate initial package.json file	package.json created with defaults	Manifest created with valid JSON structure	Pass
TC-03	Dependency	<code>npm install chalk</code>	Install Chalk package from npm registry	chalk added under dependencies	Installed chalk; node_modules created	Pass
TC-04	ESM Support	<code>"type": "module"</code>	Configure ECMAScript Module support	Enables ESM import syntax in .js	import chalk loaded without ERR_REQUIRE_ESM	Pass
TC-05	Execution	<code>npm start</code>	Run app.js and render color output	3 messages displayed in colors	Green, Blue, Yellow text rendered in terminal	Pass

9. OUTPUT

Output 1: Execution Event & Console Trace

```
[Event 1] Verified runtime environment: Node.js v25.6.1, NPM 11.9.0.
[Event 2] Executed 'npm init -y' -> Created package.json manifest.
[Event 3] Configured 'type': 'module' and added 'start': 'node app.js' script.
[Event 4] Executed 'npm install chalk' -> Downloaded Chalk ^5.6.2 and created node_modules/.
[Event 5] Created app.js with Chalk ESM imports.
[Event 6] Executed 'npm start' -> Spawned 'node app.js'.
[Event 7] Rendered Green message: 'Hello! Welcome to Node.js.'.
[Event 8] Rendered Blue message: 'This message is displayed using Chalk.'.
[Event 9] Rendered Yellow message: 'NPM successfully installed the external package.'.
[Event 10] Process terminated successfully with exit code 0.
```

Output 2: Interactive Terminal Workflow & Chalk Execution

```
jsrikar@Js-MacBook-Air NPM % node -v
v25.6.1
jsrikar@Js-MacBook-Air NPM % npm -v
11.9.0
jsrikar@Js-MacBook-Air NPM % npm init -y
Wrote to /Users/jsrikar/Documents/OBS/PBL/NPM/package.json:

{
  "name": "npm",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

jsrikar@Js-MacBook-Air NPM % npm install chalk

added 1 package, and audited 2 packages in 673ms

1 package is looking for funding
  run `npm fund` for details

found 0 vulnerabilities
jsrikar@Js-MacBook-Air NPM % npm start

> npm-demo@1.0.0 start
> node app.js

Hello! Welcome to Node.js.
This message is displayed using Chalk.
NPM successfully installed the external package.
jsrikar@Js-MacBook-Air NPM %
```

Ln 6, Col 79 Spaces: 4 UTF

9. OUTPUT (Continued)

Output 3: Manifest Verification & Package Script Run

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
jsrikar@Js-MacBook-Air NPM % cat package.json
{
  "name": "npm-demo",
  "version": "1.0.0",
  "type": "module",
  "scripts": { "start": "node app.js" },
  "dependencies": { "chalk": "^5.6.2" }
}

jsrikar@Js-MacBook-Air NPM % npm start
> npm-demo@1.0.0 start
> node app.js

Hello! Welcome to Node.js.
This message is displayed using Chalk.
NPM successfully installed the external package.

jsrikar@Js-MacBook-Air NPM %
```

10. RESULT

Thus, the complete Node Package Manager (NPM) workflow—encompassing project manifest initialization (package.json), third-party dependency installation (chalk), ECMAScript Modules (ESM) configuration ('type': 'module'), and dynamic terminal colorization—was successfully designed, executed, and verified across all test cases.

Verification Status: Completed & Tested Successfully
Execution Engine: Node.js (V8) | Package Manager: npm (Chalk v5 ESM)

Faculty Signature: _____